

ECE 757, Homework 1

Tudor Popa

February 2026

1 Profiling and analysis

To estimate the potential gains of parallelizing `kymppitonni.c`, we used the `getCC()` function to measure the execution time of each game as well as the total execution time of a run with the following parameters:

- Games per cutoff: 5000
- Minimum cutoff: 10
- Maximum cutoff: 300

This yielded the given measurements:

- $t_{total} = 273550982$ (cycles)
- $t_{parallelizable} = 271080903$ (cycles)

Using Amdahl's law, we can then estimate theoretical speedups for 1, 2, 4, 8, and 16 processors. The results were calculated using Matlab and are detailed in the graph below.

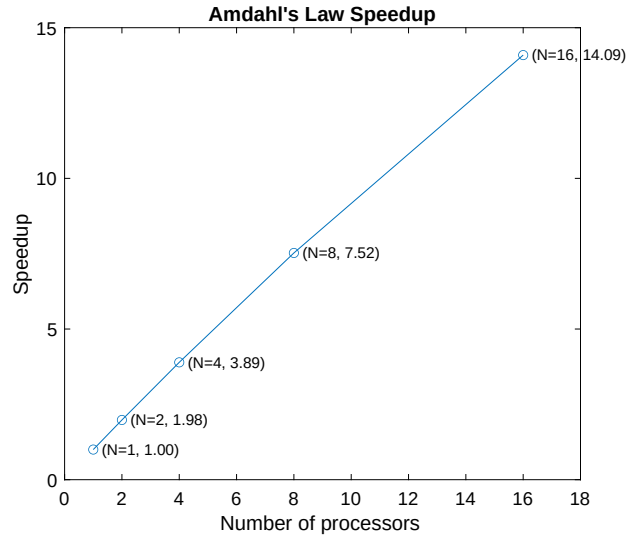


Figure 1: Theoretical Amdahl's Law Speedup

2 Running a pthreads parallelized implementation of the game

The serial version of `kymppitonni.c` was parallelized using pthreads. More specifically, for each cutoff, we ran games in parallel instead of serially. The code was modified to pass the desired number of threads to run as argument.

We successively ran it for 2-to-12 threads in increments of 1 with the same number of parameters as before; repeating it 10 times to average the execution times. The results are detailed in the graph below.

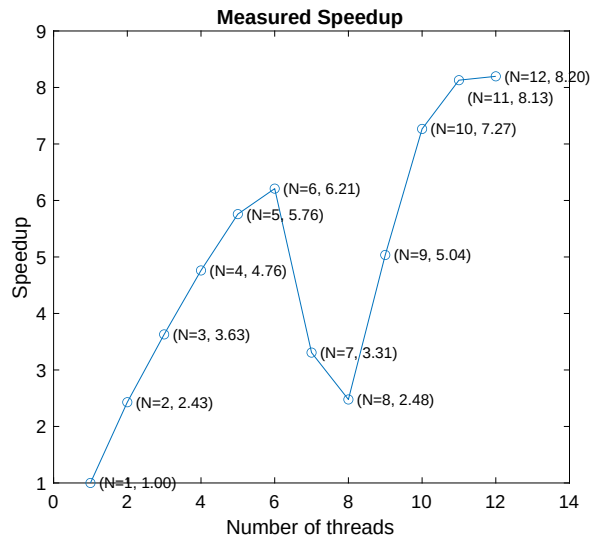


Figure 2: Speedup per number of threads (2-12)

The benchmark was run on a machine with 6 cores and 12 threads. We can observe near linear speedups up to 6 threads, as well as a dip in speedups from 7 to 10 threads. These results appear nominal, given that our test machine only possessed 6 physical threads.

3 Running an OpenMP parallelized implementation of the game

We repeated the same procedure, but using OpenMP to parallelize our program. The results are detailed on the graph below.

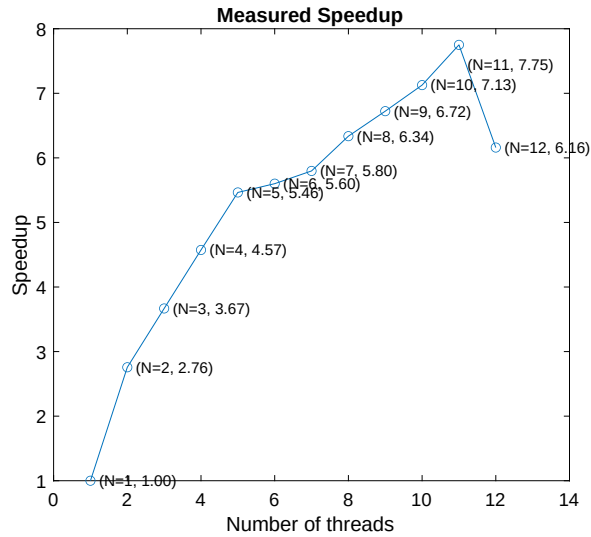


Figure 3: Speedup per number of threads (2-12)

We can observe similar results to the pthreads implementation, where linear scaling occurs up to 5 threads.